FAKE PRODUCT REVIEWS DETECTION

Using Natural Language Processing and Machine Learning

Complete Group Project Report

Group Member	Name	Roll Number	Assigned Role
Member 1	Atizaz Ahmed	L1F23BSCS0839	Dataset Collection
Member 2	Muhammad Daud	L1F23BSCS0855	NLP Preprocessing
Member 3	Nadia Aman	L1F23BSCS1299	Machine Learning Model
Member 4	Amna Kashif	L1F23BSCS0959	Evaluation and Application

Course: Introduction to Natural Language Processing (NLP)

Project topic: Detection of fake and genuine product reviews

Draft notice: Dataset-specific facts, measured model scores, confusion matrices, screenshots and repository/tag details must be filled from the group's actual executed project before submission.

1. Introduction

Online product reviews influence purchasing decisions, but some reviews may be deceptive, manipulated, incentivized or generated to mislead readers. Automatically distinguishing fake reviews from genuine reviews is a text-classification problem. This project applies Natural Language Processing (NLP) and supervised machine-learning techniques to review text, with the goal of classifying each review as fake or genuine.

The workflow includes dataset acquisition and inspection, text cleaning and normalization, feature extraction, model training, quantitative evaluation and an interactive application. The four members have separate responsibilities so that stages can be developed and checked independently.

2. Objectives

• Select and document a suitable public dataset containing product reviews and labels.

• Inspect missing values, duplicate records, class distribution and text-quality issues.

• Build a reusable text-cleaning pipeline using Unicode normalization, noise removal, tokenization and normalization.

• Represent cleaned text numerically using TF-IDF features.

• Train and compare Multinomial Naive Bayes, Logistic Regression and Support Vector Machine classifiers.

• Evaluate models using accuracy, precision, recall, F1-score and confusion matrices.

• Integrate the selected model into a Streamlit interface and document how to run it.

3. Dataset and Data Acquisition

The dataset should contain product-review text and a target label identifying each record as fake or genuine. Record the exact public source, download URL, licence, file format, number of rows, column names and label encoding from the dataset actually used.

Item	Project value
Dataset name	[Enter exact dataset name]
Source / download URL	[Enter source URL]
Licence / usage terms	[Verify licence]
Raw file format	[CSV / TSV / other]
Number of records	[Calculate]
Review-text column	[Exact column name]
Target-label column	[Exact column name]
Label mapping	[Verify fake/genuine encoding]

Provide a loading or download script and document the retrieval steps. Do not commit raw data if the course instructions prohibit it. Validate that expected text and label columns exist.

4. Data Exploration and Basic Statistics

Use pandas to report document count, missing values in relevant columns, duplicate rows, average review length in words, fake/genuine counts and percentages, vocabulary size, sample token count and type count. Inspect the token-frequency distribution and comment on whether a Zipf-like long tail is visible.

• Total documents: [measured value].

• Missing review texts / labels: [measured values].

• Duplicate records: [measured value and handling].

• Average review length: [measured words].

• Genuine reviews: [count and percentage].

• Fake reviews: [count and percentage].

• Vocabulary before cleaning: [measured value].

• Sample token count and type count: [measured values].

• Vocabulary after cleaning: [measured value].

Include three to five real, anonymized messy-text examples from the dataset. Examples may show inconsistent capitalization, repeated punctuation, HTML fragments, URLs, unusual spacing, misspellings or encoding artifacts. Do not fabricate examples.

5. Tokenization and Regular Expressions

Compare at least two tokenization methods, such as a regex-based tokenizer and NLTK's word tokenizer. Apply both to the same review sentences and show outputs side by side.

Purpose	Pattern	Explanation
URL	(?i)\\b(?:https?://|www\\.)\\S+	Finds common web links for removal or replacement.
Email	\\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}\\b	Finds email-like strings.
Whitespace	\\s+	Finds runs of whitespace for normalization.
HTML tags	<[^>]+>	Finds simple HTML tags in scraped reviews.

Test the patterns on actual records. Discuss at least two observed tokenization challenges, such as contractions, hyphenated product terms, emojis, prices, punctuation-heavy text or spelling variation.

6. Text Cleaning and Normalization Pipeline

Implement one reusable function so that every review is processed consistently. A suitable order is: (1) Unicode NFC normalization, (2) case folding/lowercasing if capitalization is not needed, (3) noise removal such as HTML tags and URLs, (4) whitespace normalization, (5) tokenization, (6) stop-word filtering while preserving negation terms such as “not” and “never”, and (7) one selected normalization method: stemming or lemmatization.

For English reviews, lemmatization preserves recognizable dictionary forms, while stemming is faster but may produce fragments. Choose one method, name the library/resources used, and justify the choice using the observed data. Preserve negation if it may signal review meaning. Retain original text and labels alongside cleaned text. Show before/after examples and calculate vocabulary sizes using the same counting method.

7. TF-IDF and Machine-Learning Models

TF-IDF converts cleaned text into numeric vectors, weighting terms by their importance in a document relative to the corpus. Fit the vectorizer on training data only and use that fitted vectorizer to transform test data to prevent information leakage.

Create a reproducible train/test split, using stratification where possible. Keep the test set separate from fitting and tuning. Train: (1) Multinomial Naive Bayes, a probabilistic text-classification baseline; (2) Logistic Regression, a linear classifier suited to sparse text features; and (3) Support Vector Machine, a margin-based classifier commonly effective for high-dimensional text. Compare all models on the same split and feature pipeline.

8. Evaluation and Results

Evaluate each model on the held-out test set. Accuracy is the proportion of correct predictions. Precision measures the proportion of predicted fake reviews that are actually fake. Recall measures the proportion of actual fake reviews identified. F1-score is the harmonic mean of precision and recall. State the positive class (recommended: fake) and whether macro or weighted averages are reported.

Model	Accuracy	Precision (fake)	Recall (fake)	F1 (fake)

Naive Bayes	[run]	[run]	[run]	[run]
Logistic Regression	[run]	[run]	[run]	[run]
SVM	[run]	[run]	[run]	[run]

Insert a confusion matrix for each model, with labelled axes and class order. Discuss false positives and false negatives. Select the best model using the metric most appropriate to the task, not accuracy alone, particularly if classes are imbalanced. Record saved model and vectorizer filenames.

Do not submit placeholder scores as results. Replace [run] with values generated by executing the final code.

9. Streamlit Application

The app should accept a product review, apply the same preprocessing pipeline used during training, transform it with the fitted TF-IDF vectorizer, load the selected saved model and display the predicted class. Handle empty input and missing model files. Explain that the output is a model prediction, not definitive proof. Test short, long and noisy inputs and include screenshots.

10. Reproducibility and Repository Organization

Use Python 3.11 or later, a virtual environment and a documented requirements file. The README should provide commands to load data, preprocess, train, evaluate and launch the app. Suggested structure:

```
fake-reviews-Assignment-1/
README.md
requirements.txt
data/README.md
notebooks/exploration.ipynb
src/data_loader.py
src/preprocess.py
src/train_models.py
src/evaluate.py
models/best_model.joblib
models/tfidf_vectorizer.joblib
app/streamlit_app.py
reports/project_report.pdf
reports/screenshots/
```

Document the exact repository URL and assignment-1 tag only after they have actually been created and pushed. Do not commit credentials or prohibited raw data.

11. Limitations and Ethical Considerations

Labels may be noisy and models may not generalize across platforms, product types, languages or time periods. Genuine reviews can be incorrectly flagged, while fake reviews can be missed. Present the app as a decision-support demonstration, not proof that a reviewer or review is dishonest. Protect personal information in examples and screenshots.

12. Conclusion

The project combines dataset inspection, NLP cleaning, TF-IDF, three supervised classifiers, evaluation and a Streamlit demonstration. Final conclusions must be based on measured results from the group's actual dataset and reproducible execution.

13. References

Cite the actual dataset and licence, pandas documentation, NLTK or spaCy documentation, scikit-learn documentation for TF-IDF and classifiers, and Streamlit documentation using the instructor's required citation style.

Final Evidence Checklist

■ Dataset source/licence verified; row counts and labels calculated.

- ■ Three to five actual messy-text examples included.
- ■ Two tokenizers compared and at least three regex patterns tested.
- ■ Cleaning order and stop-word/negation decisions justified.
- ■ Before/after examples and vocabulary counts calculated.
- ■ Cleaned output generated by script.
- ■ All three models trained and evaluated on the same split.
- ■ Real metrics and confusion matrices inserted.
- ■ Best model and vectorizer loaded by Streamlit.
- ■ Application screenshots and test evidence inserted.
- ■ README run commands, requirements and exact repository/tag link verified.